

DataBridge

Full Product Summary
Written for LLM context — feature brainstorming & product positioning

What Is DataBridge?

DataBridge is an AI-powered semantic data platform built for **small and medium businesses (SMBs) that don't have a dedicated data team**. It bridges the gap between raw business data and the people who need to understand it — letting non-technical users connect their databases, ask questions in plain English, build dashboards, write AI-assisted notebooks, and monitor data quality — without hiring a data engineer or writing a line of SQL.

The platform has three primary layers:

- **The Semantic Layer** — an AI-assisted business dictionary mapping technical column names to human-readable meanings, relationships, and KPI definitions.
- **The Intelligence Layer** — Claude (Anthropic) converts natural language to SQL, designs star schemas, generates dashboards, explains results, and identifies knowledge gaps.
- **The Data Layer** — a Delta Lake warehouse where ingested data lands, is modelled into star schemas, profiled for quality, and made queryable.

Who Is It For?

Target market: SMBs with 10–500 employees that have valuable data in their operational systems but no in-house data team. Think: a regional retailer, a growing SaaS company, a professional services firm, or a manufacturing SMB running an ERP with years of untapped data.

Role	Who	What they do
Admin	IT / consultants / data-savvy staff	Connect sources, approve AI definitions, manage KPIs, invite team
Analyst	Power users	Ask complex questions, build dashboards, design data products, use notebooks
Viewer	Business users	Consume dashboards, ask simple questions, see plain-language answers

The Problem DataBridge Solves

Most SMB data sits in operational databases (ERP, CRM, accounting, HR) that are technically complex, inaccessible to non-technical staff, fragmented across systems, and ungoverned. SMBs can't afford a data team, a BI consultant, or an enterprise platform. DataBridge puts an AI-assisted semantic layer on top of raw data and handles the complexity — so the gap between *"the data exists"* and *"the business can use it confidently"* is closed without headcount.

Full User Journey

Step 1 — Connect Data Sources (Admin)

- Picks a connector: SQLite, PostgreSQL, MySQL, or SQL Server (Excel, Exact Online, Odoo, Salesforce, Google Sheets coming soon)
- Enters credentials, assigns data domains (tags like 'sales', 'finance', 'hr'), tests and saves

Step 2 — AI Schema Profiling (Automatic)

- Schema discovery — reads all tables, columns, data types, row counts, sample values
- Quality profiling — null rate, distinct rate, min/max, histograms per column

- AI draft generation — Claude writes descriptions, guesses KPIs, suggests foreign keys, flags dimensions vs. measures
- Semantic storage — saved to Postgres + Neo4j knowledge graph
- All AI content flagged `ai_draft: true` until a human approves

Step 3 — Curate the Semantic Layer (Admin)

- Reviews AI-drafted table/column descriptions in the data dictionary UI
- Edits, approves, or rejects each definition
- Defines KPIs with formulas (written manually or generated from plain English by Claude)
- Sets up cross-source relationships via a visual RelationshipCanvas graph

Step 4 — Ingest Data into the Warehouse (Admin)

- Triggers the IngestionWizard — selects tables for full or incremental (watermark CDC) load
- Python ETL service (FastAPI + PyArrow) writes Delta Lake Parquet files
- Supports local disk and Azure Blob Storage — progress streamed live per-table

Step 5 — Design Star Schema Data Products (Admin / Analyst)

- Creates a Data Product — a Kimball dimensional model from ingested tables
- Claude designs fact tables, dimension tables, and DuckDB transformation SQL
- `dim_date` calendar dimension always auto-injected
- Admin reviews schema in ReactFlow diagram, edits SQL, triggers transformation execution
- Quality gates check surrogate key uniqueness and fan-out ratios
- Transformations schedulable via cron (BullMQ)

Step 6 — Ask Questions in Natural Language (All Users)

- Types a question: "Who are my top 10 customers by revenue in Q1?"
- Claude builds semantic context from Neo4j, generates SQL + confidence score
- Entity pre-flight check: validates string literals against actual dimension values
- Confidence ≥ 0.70 → SQL executes, Claude writes a plain-language answer
- Confidence < 0.70 → blocked, logged as definition gap, graceful message to user
- Suspicious result → SSE repair loop (up to 5 turns) reconstructs SQL
- Multi-turn conversation, auto-charts, data tables, forecasts with confidence bands

Step 7 — Explore Data in Notebooks (Analyst / Admin)

- SQL cells and Python cells side by side
- Pre-wired to DuckDB: source tables as `connection.table`, product tables by product name
- AI code assistant: describe what you want in plain English, Claude writes the SQL or Python
- Results display inline — useful for ad-hoc analysis and complex queries beyond chat

Step 8 — Build AI Dashboards (All Users)

- Describe dashboard in natural language; optional refinement loop with 3–4 clarifying questions
- Claude generates Dashboard Spec: KPI cards, bar/line/pie/treemap charts, tables, filters, SQL
- Cross-filtering (click a bar → filters all other widgets), date pickers, multi-select dropdowns
- Auto-refresh toggle, PDF export, URL-based sharing (viewers get read-only access)

Step 9 — Monitor Data Quality (Admin / Analyst)

- Health rings show per-table scores: completeness, uniqueness, validity
- Quality rules: null check, range, uniqueness, format regex, freshness, custom SQL
- Failures captured with offending record, field, and actual value

- Auto-alerts when score drops > 10% or falls below 75%

Step 10 — Govern and Monitor (Admin)

- Gaps page: every blocked question listed — knows exactly where to add definitions
- Query log: full history with confidence scores per question
- Team management: invite by email, assign admin / analyst / viewer roles
- Approval workflows with configurable auto-approval delay
- Audit trail: every definition change tracked with author and timestamp

AI Usage — How Claude Is Used

Call Type	Triggered When	What Claude Does
Schema Draft	After source connection	Writes descriptions for every table + column; suggests FK relationships; flags dimensions
NL → SQL	Every user question	Generates SQL + confidence score; supports SQLite, Postgres, MySQL, MSSQL, Snowflake
Notebook Assistant	User describes code to write	Writes SQL or Python cell code on request
Repair Loop	Suspicious query result	5-turn SSE agentic loop: runs diagnostics, rewrites SQL
Answer Formatting	After query executes	Writes 1-3 plain-language sentences summarising the result
Dashboard Generation	User requests a dashboard	Generates full JSON spec (widgets, filters, SQL) from natural language
Dashboard Refinement	Before/after generation	Asks clarifying questions; iterates on existing spec
Star Schema Design	Admin creates a data product	Designs fact/dimension tables + DuckDB transformation SQL
Report Narrative	User generates a report	Writes executive summary tying KPI results together

Key Design Principles

- **Never show SQL to business users.** Hidden by default; a toggle reveals it only for admin/analyst roles.
- **Confidence gating.** Queries below 0.70 are blocked and silently logged — users get a graceful message, never a hallucinated answer.
- **AI drafts everything, humans approve.** All AI content is flagged ai_draft: true; visible and usable, but approval is always a human action.
- **Semantic layer is the source of truth.** Claude never queries raw schema alone — it always gets enriched semantic context first.
- **Graceful degradation.** Redis down? Jobs run inline. Neo4j down? Products still work. Always operational.
- **Multi-tenancy by default.** Every table has tenant_id; Postgres RLS enforces isolation at the DB layer regardless of application code.
- **Deterministic fixes before AI fixes.** SQL auto-repair uses heuristics (fuzzy column matching) before ever calling Claude again.

Complete Feature Set

- Multi-connector data sources (SQLite, Postgres, MySQL, SQL Server)
 - AI schema profiling with streaming progress UI
 - Full semantic data dictionary (tables, columns, KPIs, relationships)
 - Quality profiling (per-field stats, histograms, rules, alerts, trends)
 - ETL ingestion to Delta Lake (full + incremental watermark CDC)
 - Star schema data products (AI-designed + DuckDB-materialized)
 - Cron-based transformation scheduling (BullMQ)
 - Natural language query with confidence gating, entity pre-flight, repair loop
 - Multi-turn conversation with full history
 - AI-assisted notebooks (SQL + Python, DuckDB-backed, AI writes the code)
 - AI dashboard builder with cross-filtering, drill-down, PDF export, sharing
- Executive report builder with KPI charts and AI narrative
 - Definition gap tracking and query log
 - Multi-tenancy (Postgres RLS, JWT, per-tenant data isolation)
 - Role-based access (admin / analyst / viewer)
 - Team management, email invites, role assignment
 - Approval workflows for AI-generated definitions
 - Audit trail and change history for all definitions
 - In-app notifications (jobs, quality alerts, invites)
 - Background job queue with BullMQ + Redis (optional)
 - Azure production deployment (Container Apps, Blob Storage, Key Vault)
 - CI/CD via GitHub Actions + Terraform IaC

What DataBridge Is NOT

- **Not for enterprise** — sweet spot is SMBs with no dedicated data function
- Not a pixel-perfect BI report builder (no drag-and-drop layout editor)
- Not a real-time streaming platform (batch ingestion, not event streaming)
- Not a replacement for Snowflake / BigQuery / Databricks at enterprise scale

Technology Fingerprint

Layer	Technology
AI	Claude (claude-sonnet-4-6), Anthropic SDK
Backend API	Node.js 20+, TypeScript, Express
Semantic DB	PostgreSQL 16 with Row-Level Security
Knowledge Graph	Neo4j 5 Community
Transformation Engine	DuckDB
Warehouse Format	Delta Lake (Apache Parquet)
ETL Service	Python 3.12, FastAPI, PyArrow
Job Queue	BullMQ + Redis
Frontend	Next.js 14 (App Router), Tailwind CSS, Recharts, ReactFlow
Auth	JWT + bcrypt
Storage	Local filesystem / Azure Blob Storage
Infrastructure	Docker Compose (dev), Azure Container Apps (prod), Terraform
CI/CD	GitHub Actions
Testing	Vitest (backend unit), Playwright (E2E)